7. DynamoDB

1. NoSQL

1.1 NoSQL

NoSQL

- Non SQL 또는 Non Relational을 나타내는 말
- 즉, 관계가 없는 데이터베이스를 의미

NoSQL 데이터베이스 특징

- NoSQL 데이터베이스는 관계형 데이터베이스의 데이터 일관성 제약을 일부 완화함으로써, 수평적으로 확장 가능한 성능을 갖고 있다.
- 개발이 용이하며 짧은 지연 시간 및 복원력을 갖고 잇다. → 스키마 없는 데이터 모델에 최적화되어 있다.
- 간단한 데이터 구조를 갖고 있어 열, 문서, 그래프, 인메모리 키-값 스토어 등의 다양한 데이터 모델을 사용할 수 있다. → 빅데이터, 모바일 및 웹 애플리케이션에 사용하기에 매우 적합

1.2 RDBMS vs NoSQL

	RDB(SQL)	NoSQL
데이터 저장 모델	table	Json document / key-value / 그래프 등
개발 목적	데이터 중복 감소	애자일 / 확장가능성 / 수정가능성
예시	Oracle, MySQL, PostgreSQL 등	MongoDB, DynamoDB 등
Schema	엄격한 데이터 구종	유연한 데이터 구조
장점	- 명확한 데이터 구조 보장 - 데이터 중복 없이 한 번만 저장(무결성) - 데이터 중복이 없어서 데이터 update 용이	- 유연하고 자유로운 데이터 구조 - 새로운 필드 추가 자유로움 - 수평적 확장(Scale out) 용이
단점	- 시스템이 커지면 Join문이 많은 복잡한 query - 수평적 확장이 까다로워 비용이 큰 수직적 확장(Scale up)이 주로 사용 됨.	- 데이터 중복 발생 가능 - 중복 데이터가 많기 때문에 데이터 변경 시 모든 컬렉션에서 수정 필요 - 명확한 데이터 구조 보장 X
사용	- 데이터 구조가 변경될 여지 없이 명확한 경우 - 데이터 update가 잦은 시스템	- 정확한 데이터 구조가 정해지지 않은 경우 - Update가 자주 이뤄지지 않는 경우 - 데이터 양이 매우 많은 경우

<Note> **ACID 속성**?

- 원자성(Atomicity): 트랜잭션이 완전히 실행되거나 전혀 실행되지 않는 '모두 실행 또는 전혀 실행되지 않음'을 의미
- 일관성(Consistency): 트랜잭션이 커밋되면 데이터가 데이터베이스 스키마에 준수해야 함을 의미
- 격리성(Isolation): 동시에 일어나는 트랙잭션들이 각기 별도로 실행되어야 함을 의미
- 내구성(Durability): 예기치 못한 시스템 장애 또는 정전 시 마지막으로 알려진 상태로 복구하는 기능

1.3 NoSQL 데이터베이스 유형

4가지 유형

- 열 기반 데이터베이스: 데이터의 행이 아닌 데이터의 열을 읽고 쓰는 데 최적호ㅏ
- 문서 데이터베이스: JSON 또는 XML 형식의 문서와 같은 반정형 데이터를 저장하도록 설계
- 그래프 데이터베이스: 간선이라고 부르는 방향 링크와 정점을 저장하는 형태의 데이터베이스
- 인 메모리 키-값 스토어: 읽기 중심의 애플리케이션 워크 로드 또는 컴퓨팅 집약적 워크로드에 최적화 되어 있다.

---

2. DynamoDB

DynamoDB

- AWS에서 제공하는 매우 빠르고 확장 가능한 완전 관리형 클라우드 NoSQL 데이터베이스
- DAX(Amazon DynamoDB Accelerator)라는 것이 있는데 이것은 가용성이 뛰어난 완전 관리형 인 메모리 캐시다.

DynamoDB의 Read 방식

- **Eventually Consistent Read**
  - 기본으로 사용되는 방식
  - 최근 완료된 쓰기 작업의 결과를 반영하지 않을 수 있기 때문에 부실 데이터가 일부 포함될 수 있는 방식

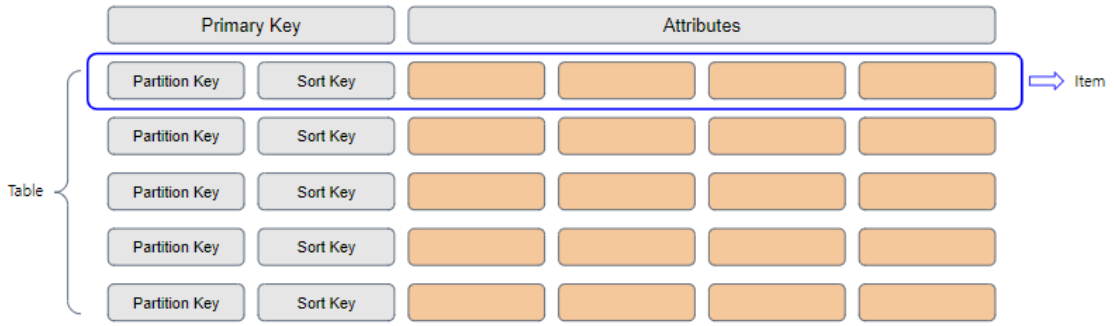
- **Strongly Consistent Read**
  - 성공한 모든 이전 쓰기 작업의 업데이트를 반영하여 가장 최신 데이터로 응답을 반환하는 방식
  - 네트워크 지연 또는 중단이 발생한 경우에 사용이 어려울 수 있다.

DynamoDB의 과금 방식

- **스토리지 요금**
  - 테이블에서 사용한 디스크 공간(GB)에 따른 시간당 요금을 부과
- **읽기/쓰기 용량 유닛에 대한 요금**
  - 1 읽기 용량 유닛은 초당 최대 2건의 읽기 작업을 제공하며, 1 쓰기 용량 유닛은 초당 최대 1건의 쓰기작업을 제공

<u>DynamoDB의 기본 구성</u>

- **Table**: Item의 집합
- **Item**: Attribute의 집합
- **Attribute**: Key-Value 방식의 데이터(Key는 문자열)



---

3. DynamoDB 데이터 타입

스칼라 데이터 형식: 하나의 값만 표현

- Number(N)
- String(S)
- Binary(B)
- Boolean(S, 0 또는 1)

- Null(NULL)

문서 형식: 내포 속성이 있는 복잡한 구조를 표현

- List(L)
- Map(M)

다중 값 형식: 여러 스칼라 값을 표현

- String Set(SS)
- Number Set(NS)
- Binary Set(BS)

---

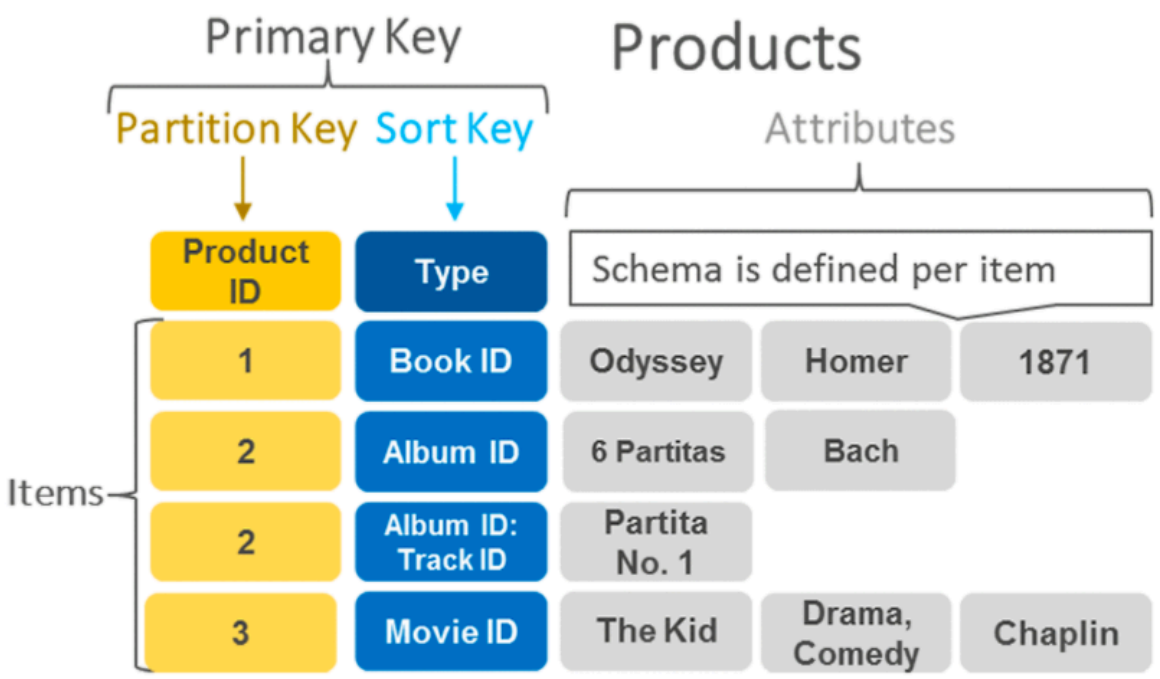
4. DyamoDB 파티션 및 키

DynamoDB의 파티션

- DynamoDB가 데이터를 저장하는 곳

DynamoDB의 키

- **기본키(Primary Key)**
  - Table 내에서 각 Item을 구분하는 고유 식별자
  - **단순 기본 키**: 파티션 키만 사용
  - **복합 기본 키:** 파티션 키와 정렬 키를 함께 사용
- **파티션 키(Partition Key)**
  - 내부 해시 함수에 대한 입력으로 파티션 키 값을 사용
  - 출력에 따라 항목을 저장할 파티션이 결정됨
- **정렬 키(Sort Key)**
  - 파티션 키가 동일한 모든 항목을 정렬하는 키 값



---

5. DynamoDB 보조 인덱스

DynmoDB의 테이블에서는 하나 이상의 보조 인덱스를 생성할 수 있다.

- 보조 인덱스는 파티션 키에 대한 쿼리는 물론이고 정렬 키를 사용하여 테이블 데이터에 대한 쿼리까지 실행 할 수 있다.

보조 인덱스의 종류

- 로컬 보조 인덱스
  - Local Secondary Index(LSI)
  - 테이블과 파티션 키는 동일하지만 정렬 키는 다른 인덱스다.
  - 테이블에 정렬 키를 추가로 더 만든다고 생각하면 된다.
- 글로벌 보조 인덱스
  - Global Secondary Index(GSI)
  - 파티션 키 및 정렬 키가 테이블의 파티션 키 및 정렬 키와 다를 수 있는 인덱스
  - 테이블당 최대 20개의 GSI 및 5개의 LSI를 정의할 수 있다.

특성	글로벌 보조 인덱스	로컬 보조 인덱스
키 스키마	단순 기본 키(파티션 키) 또는 복합 기본 키(파티션 키 및 정렬 키)	기본 키는 반드시 복합 기본 키(파티션 키 및 정렬 키)여야 함
키 속성	문자열, 숫자 또는 이진수 형식의 기본 테이블 속성	기본 테이블의 파티션 키와 동일한 속성
파티션 키 값당 크기 제한	크기 제한이 없음	파티션 키 값마다 인덱싱된 모든 항목의 전체 크기가 10GB 이하여야 함
온라인 인덱스 작업	테이블을 생성할 때 동시에 GSI를 생성할 수 있음. 기존 테이블에 새 GSI를 추가하거나, 기존 GSI를 삭제할 수도 있음	테이블을 생성할 때 동시에 LSI가 생성됨. 기존 테이블에 LSI를 추가할 수도 없고, 기존 LSI를 삭제할 수도 없음
쿼리 및 파티션	전체 테이블의 모든 파티션에 대해 쿼리 가능	파티션키가 같은 단일 파티션에 대해 쿼리 가능
읽기 일관성	최종 일관성만 지원	최종 일관성 또는 강력한 일관성 선택 가능
할당된 처리량 소비	자체 할당 처리량 설정이 있음 GSI의 용량 단위를 소비	기본 테이블의 읽기/쓰기 용량 단위를 소비 테이블에 쓸 때 해당 LSI도 업데이트 됨
프로젝션 속성	인덱스로 프로젝션되는 속성만 요청할 수 있음	인덱스로 프로젝션되지 않는 속성도 요청할 수 있음

---

6. DynamoDB 데이터 조회

DynamoDB의 데이터 조회 방식

- **Query**
  - 기본 키 값을 기반으로 항목을 찾는 방식
  - 키 값을 찾는 것
  - 예: 학번으로 곧바로 부르는 것
- **Scan**
  - 테이블 또는 보조 인덱스의 모든 항목을 읽어오는 방식
  - 전체 데이터를 가져와서 필터링하는 것
  - 예: 한 명씩 불러서 학번이 일치하는지 확인하는 것
- 데이터를 조회할 때는 되도록 Query를 사용하는 것이 좋고 Scan은 속도도 느리고 비싸므로 가급적 사용하지 않는 것이 좋다.

---

7. DynamoDB 실습 - 테이블 설계

전교생의 시험 점수를 DynamoDB에 넣어서 관리하려고 한다.

- 조건

    - 학생들은 저마다 고유으 학번을 갖고 있다.
    - 시험 과목은 영어, 과학, 수학 총 3과목
    - 데이터 조회가 가능해야 한다.
        - 한 학생의 영어, 과학, 수학 점수를 다 가져오기
        - 각 과목별 점수로 정렬하여 학생 목록 가져오기

7.1 기본 키를 어떻게 구성해야 할까

기본 키(Primary Key)

- 단순 기본 키로 할 경우
    - 파티션 키: 학생의 학번
    - 요구사항에 있었던 것처럼 각 과목별 점수로 정렬하여 학생 목록을 가져올 수 없다.
    - 조건을 만족시키려면 복합 기본 키를 사용하고 보조 인덱스를 추가해야 한다.

학번	영어	과학	수학
12093	80	92	95

- 복합 기본 키로 할 경우
    - 파티션 키: 학생의 학번
        - 정렬 키: 과목명

학번	과목	점수
	영어	80
12093	과학	92
	수학	95

7.2 보조 인덱스 추가하기

- 글로벌 보조 인덱스
    - 복합 기본 키
        - 파티션 키: 과목 키
        - 정렬 키: 점수

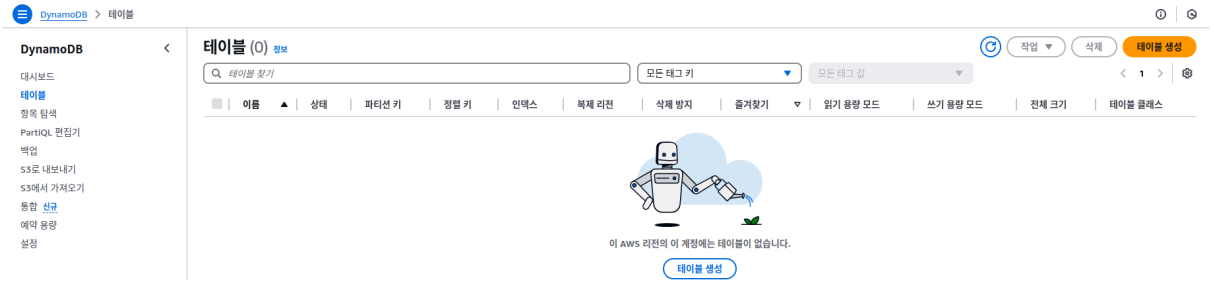
학번	점수	점수

	100	12103
영어	98	12119
	96	12067
	100	12048
과학	99	12091
	93	12758
	96	12746
수학	90	12342
	88	12473

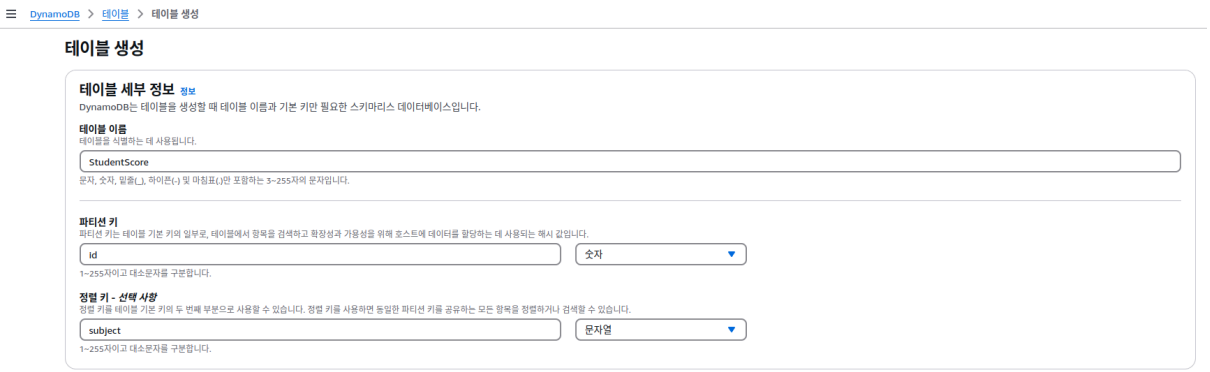
8. DynamoDB 테이블 생성

- **DynamoDB > 테이블 생성**

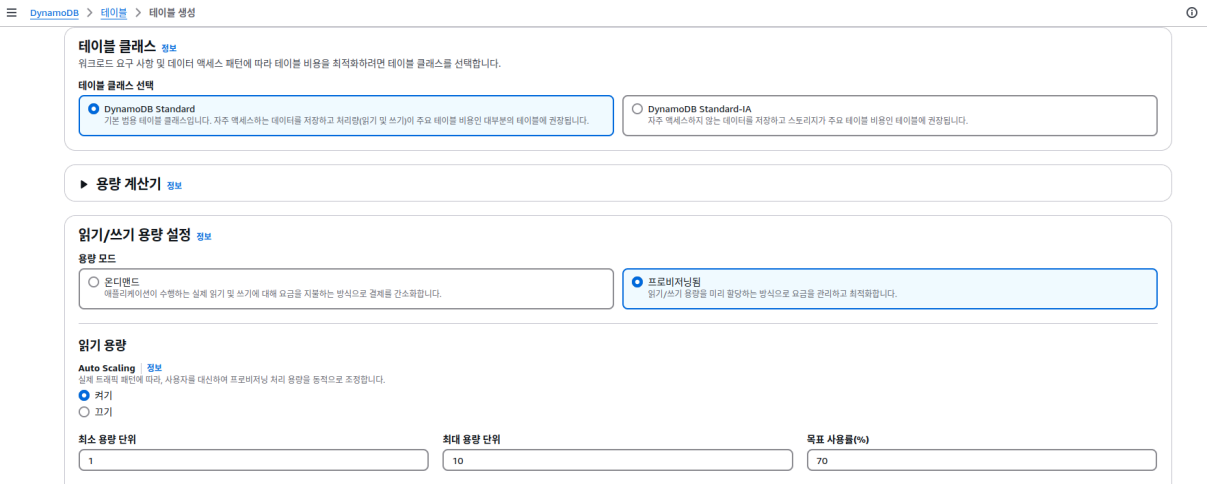
- **테이블 > 테이블 생성**



- **테이블 생성**
  - 테이블 이름: StudentScore
  - 파티션 키: id
  - 정렬 키: 숫자



- **테이블 생성**
  - **설정 사용자 지정**: 기본 설정 그대로
  - **읽기/쓰기 용량 설정**: 프로비저닝 - 기본 설정 그대로



- **보조 인덱스 > 글로벌 인데스 생성 > 새 글로벌 보조 인덱스**
  - 파티션 키: subject(id 아님) - 문자열 ← 과목명으로 파티션을 나눌 것이기 때문
  - 정렬 키: score - 숫자

새 글로벌 보조 인덱스

글로벌 보조 인덱스를 생성하여 원래 테이블의 기본 키 외부의 속성을 쿼리합니다. 자세히 알아보기

파티션 키

subject

1~255자.

데이터 형식

문자열

정렬 키 - *선택 사항*

score

1~255자.

데이터 형식

숫자

인덱스 이름

subject-score-index

3~255자의 문자입니다. A~Z, a~z, 0~9, 밑줄 문자, 하이픈 및 마침표만 허용됩니다.

속성 프로젝션

프로젝션이란 테이블에서 보조 인덱스로 복사되는 속성 집합입니다.

- (●) **All**
  모든 테이블 속성이 인덱스에 프로젝션됩니다.
- ( ) **Only keys**
  인덱스와 기본 키만 인덱스에 프로젝션됩니다.
- ( ) **Include**
  "키만 보기"에 설명된 모든 속성과 키 외에 사용자가 지정한 다른 속성입니다.

▶ **웜 처리량**

웜 처리량 값을 높이면 테이블을 미리 예열하여 스로틀링이나 확장 지연 없이 계획된 피크 이벤트를 처리할 수 있습니다. 기본적으로 웜 처리량 값은 모든 테이블과 글로벌 보조 인덱스에 대해 표시됩니다. 이 값은 프로비저닝된 처리량 또는 온디맨드 소비량이 증가하면 추가 요금 없이 자동으로 조정되지만, 수동으로 변경하는 경우에는 추가 요금이 적용됩니다.

ⓘ 기본적으로 글로벌 보조 인덱스의 용량은 기본 테이블의 용량과 동일합니다. 테이블을 생성한 후 테이블의 설정에서 인덱스 용량을 변경할 수 있습니다.

취소 | 인덱스 생성

- **보조 인덱스 생성 확인**

보조 인덱스 정보

삭제 | 로컬 인덱스 생성 | 글로벌 인덱스 생성

이름	유형	파티션 키	정렬 키	프로젝션 속성	웜 처리량
subject-score-index	글로벌	subject (String)	score (Number)	모두	-

- **테이블 생성 확인**

DynamoDB > 테이블

DynamoDB

대시보드
테이블
항목 탐색
PartiQL 편집기
백업
S3로 내보내기

StudentScore 테이블이 생성되었습니다.

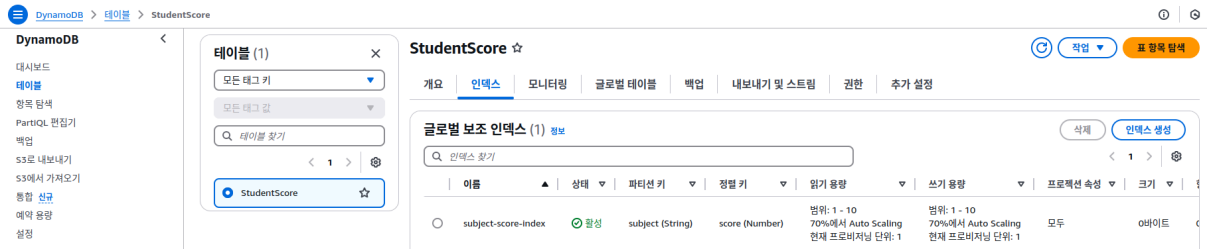
테이블 (1) 정보

작업 | 삭제 | 테이블 생성

테이블 찾기 | 모든 태그 키 | 모든 태그 값

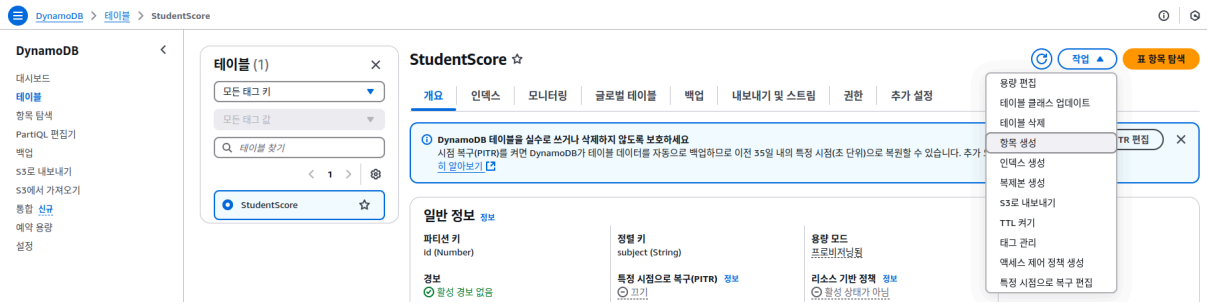
이름	상태	파티션 키	정렬 키	인덱스	복제 리전	삭제 방지	즐겨찾기	읽기 용량 모드	쓰기 용량 모드	전체 크기	테이블 클래스
StudentScore	활성	id (N)	subject (S)	1	0	끄기		프로비저닝됨 (1)	프로비저닝됨 (1)	0바이트	표준

- **테이블 상세 정보 > 인덱스 탭**
  - 글로벌 보조 인덱스 확인



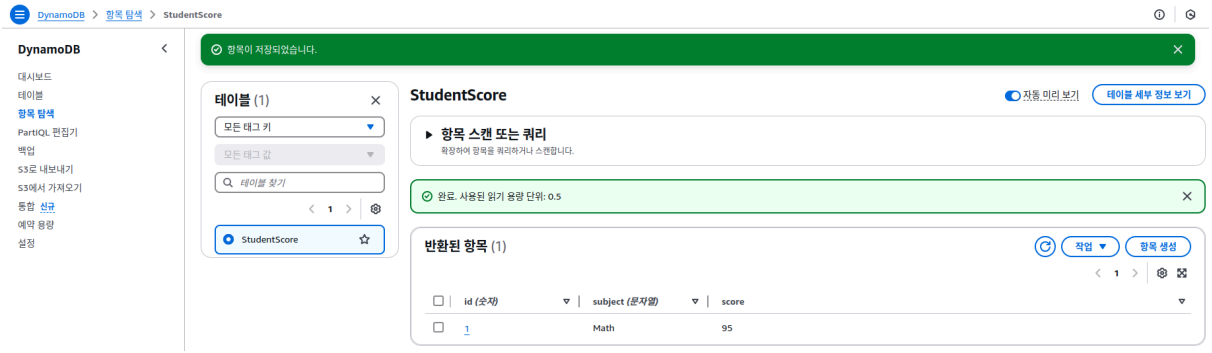
9. DynamoDB 데이터 입력

- **테이블 상세 페이지 > 작업 > 항목 생성**

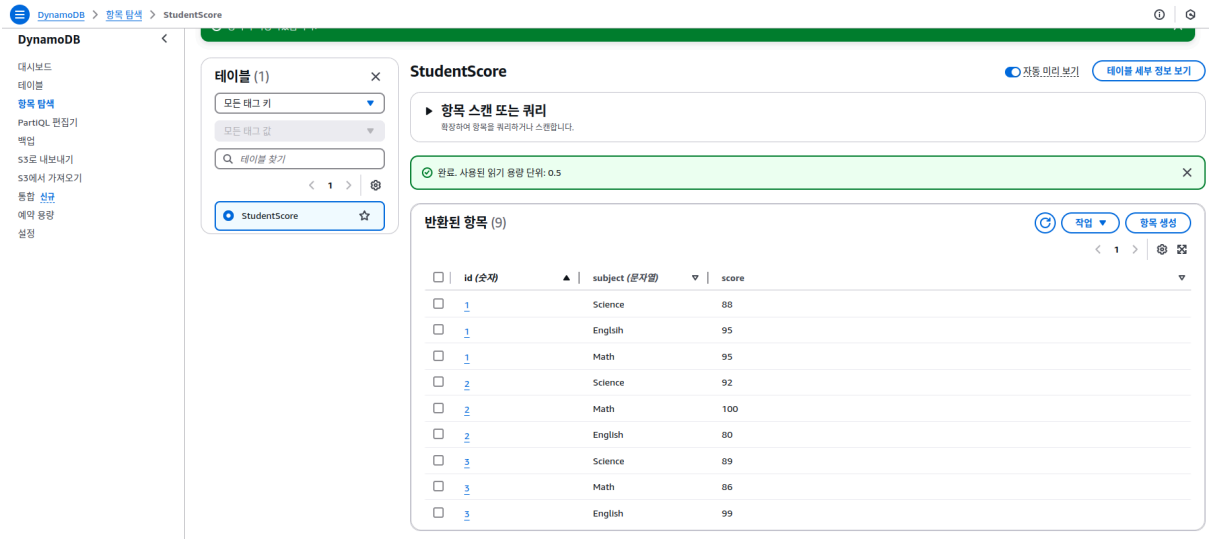


- **항목 생성: 1, Math, 95**
  - **새 속성 추가: score**

- **항목 확인**


DynamoDB > 항목 탐색 > StudentScore
DynamoDB
대시보드
테이블
항목 탐색
PartiQL 편집기
백업
S3로 내보내기
S3에서 가져오기
통합 신규
예약 용량
설정
항목이 저장되었습니다.
테이블 (1)
모든 태그 키
모든 태그 값
테이블 찾기
StudentScore
StudentScore
자동 미리 보기
테이블 세부 정보 보기
항목 스캔 또는 쿼리
확장하여 항목을 쿼리하거나 스캔합니다.
완료. 사용된 읽기 용량 단위: 0.5
반환된 항목 (1)
작업
항목 생성
id (숫자)
subject (문자열)
score
1
Math
95

- **동일한 과정: 총 9번 반복**

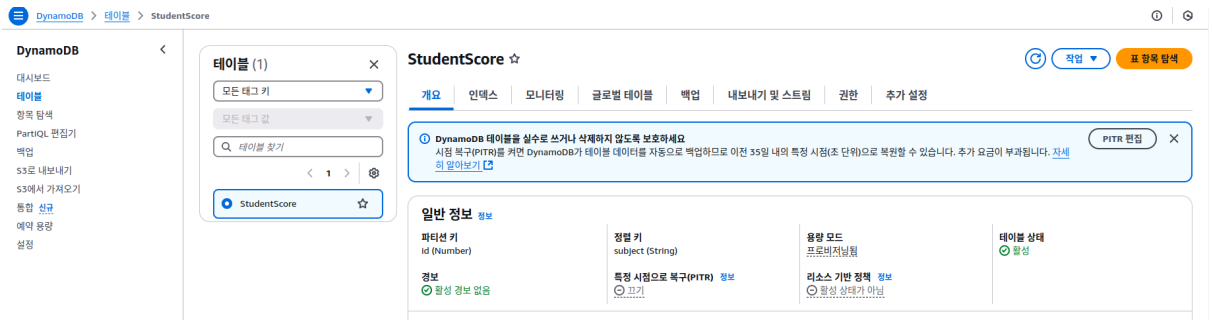

DynamoDB > 항목 탐색 > StudentScore
DynamoDB
대시보드
테이블
항목 탐색
PartiQL 편집기
백업
S3로 내보내기
S3에서 가져오기
통합 신규
예약 용량
설정
테이블 (1)
모든 태그 키
모든 태그 값
테이블 찾기
StudentScore
StudentScore
자동 미리 보기
테이블 세부 정보 보기
항목 스캔 또는 쿼리
확장하여 항목을 쿼리하거나 스캔합니다.
완료. 사용된 읽기 용량 단위: 0.5
반환된 항목 (9)
작업
항목 생성
id (숫자)
subject (문자열)
score
1 Science 88
1 Englsih 95
1 Math 95
2 Science 92
2 Math 100
2 English 80
3 Science 89
3 Math 86
3 English 99

반환된 항목 (9)

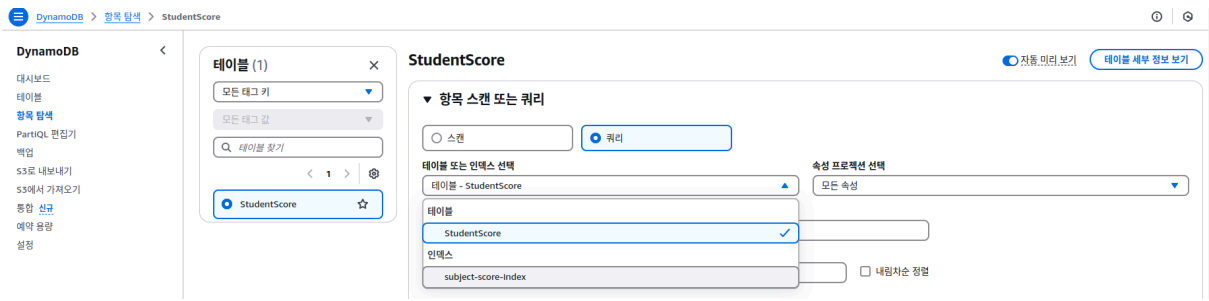
id (숫자)	subject (문자열)	score
1	Science	88
1	Englsih	95
1	Math	95
2	Science	92
2	Math	100
2	English	80
3	Science	89
3	Math	86
3	English	99

10. DynamoDB 데이터 조회

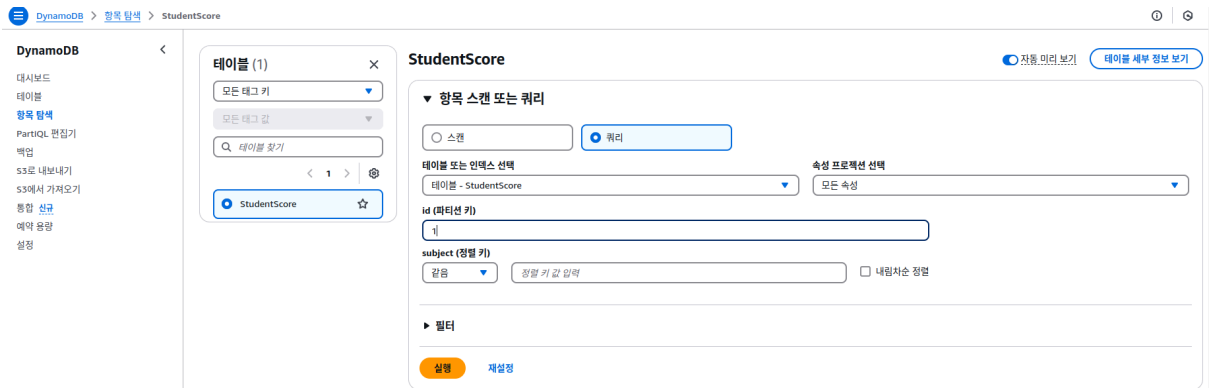
- 테이블 상세 페이지 > 표 항목 탐색



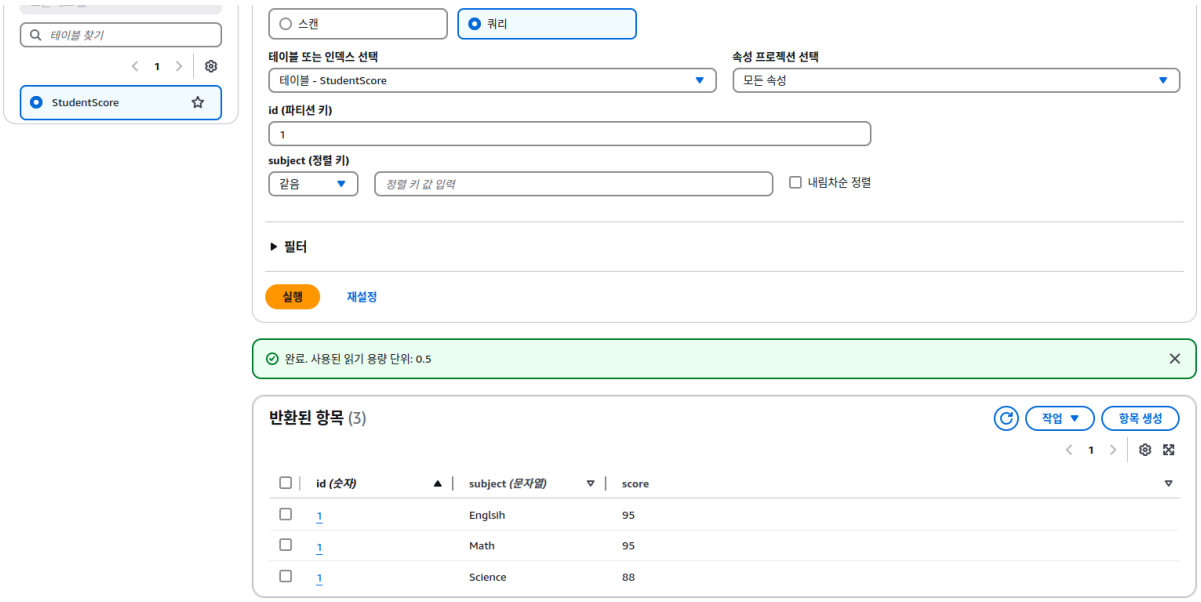
- **쿼리 선택(스캔 또는 쿼리)**
  - 테이블의 기본 키로 조회



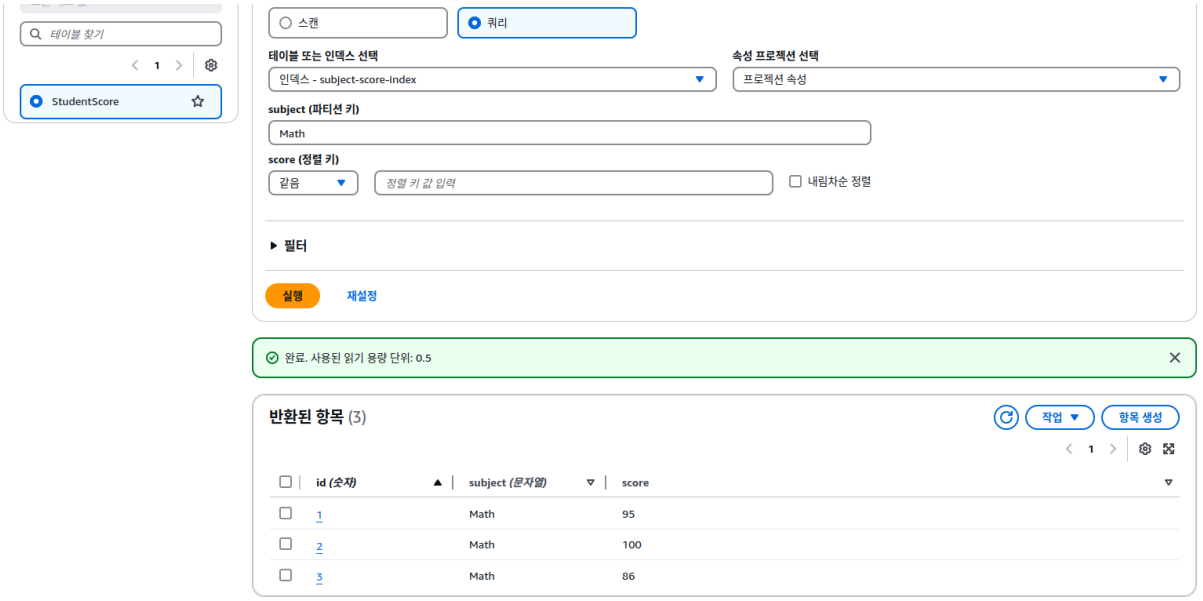
- **id 1 → 실행**



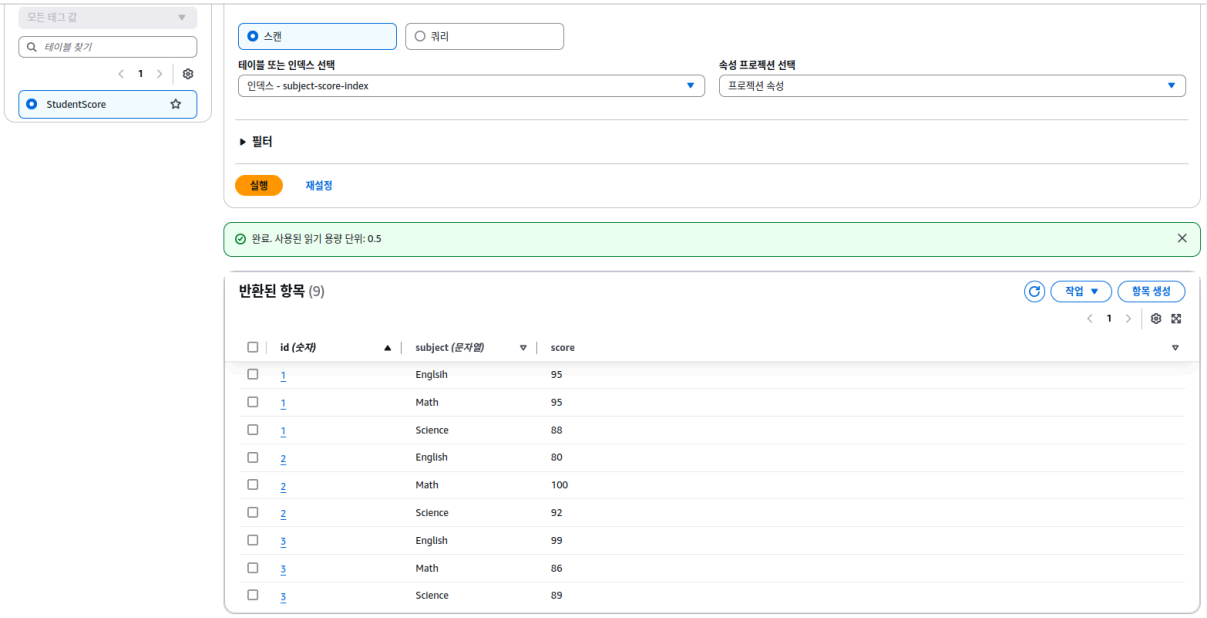
- **1번 학생의 전과목에 대한 정보**



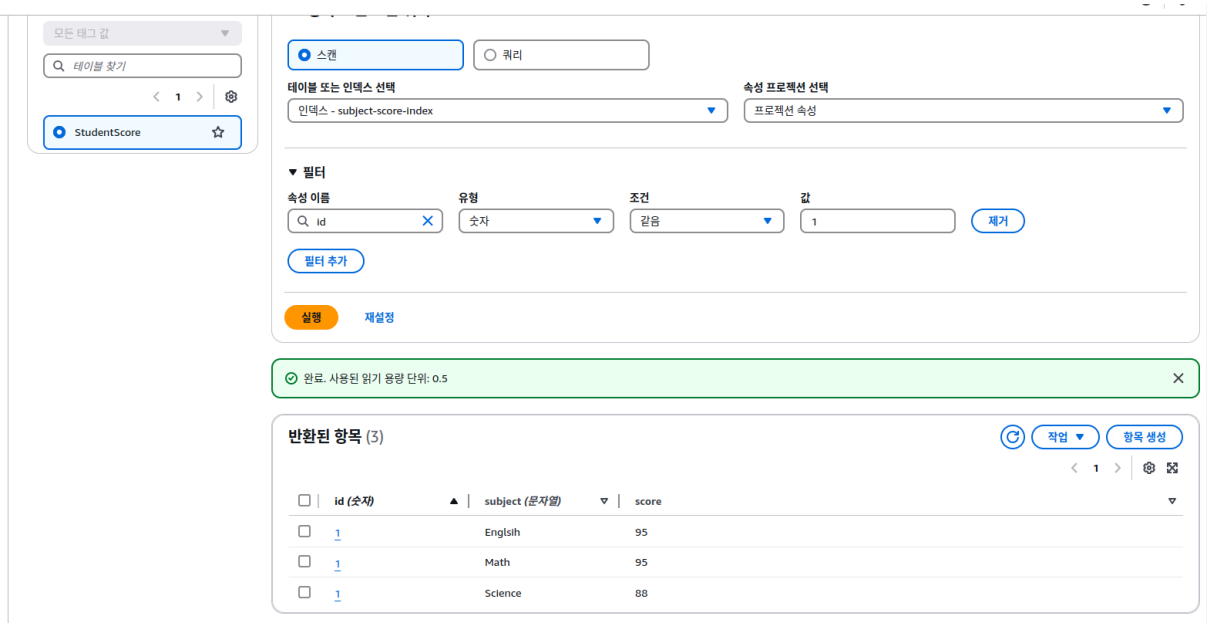
- **글로벌 보조 인덱스**
  - 과목 조회: Math



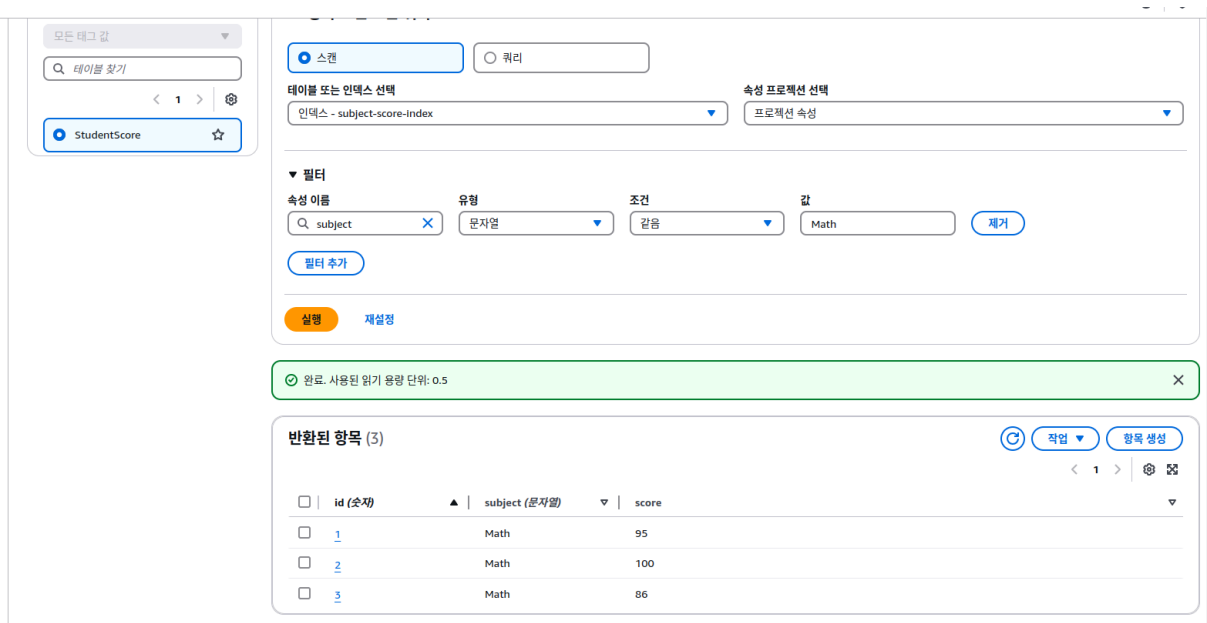
- **스캔**
  - 전체 데이터 조회



- **스캔: 필터 추가**
  - 속성 이름: id
  - 유형: 숫자
  - 값: 1
  - 1번 학생의 모든 과목 정보 조회



- **Scan: 모든 학생의 수학 과목 정보 조회**
  - 속성 이름: subnet
  - 유형: 문자열
  - 값: Math

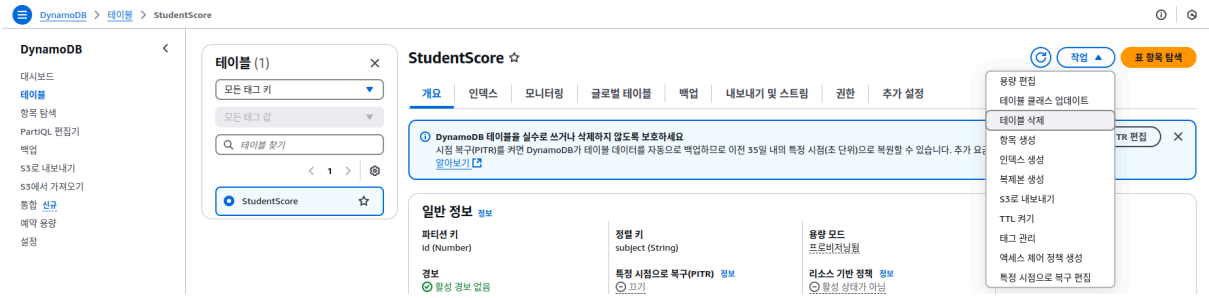
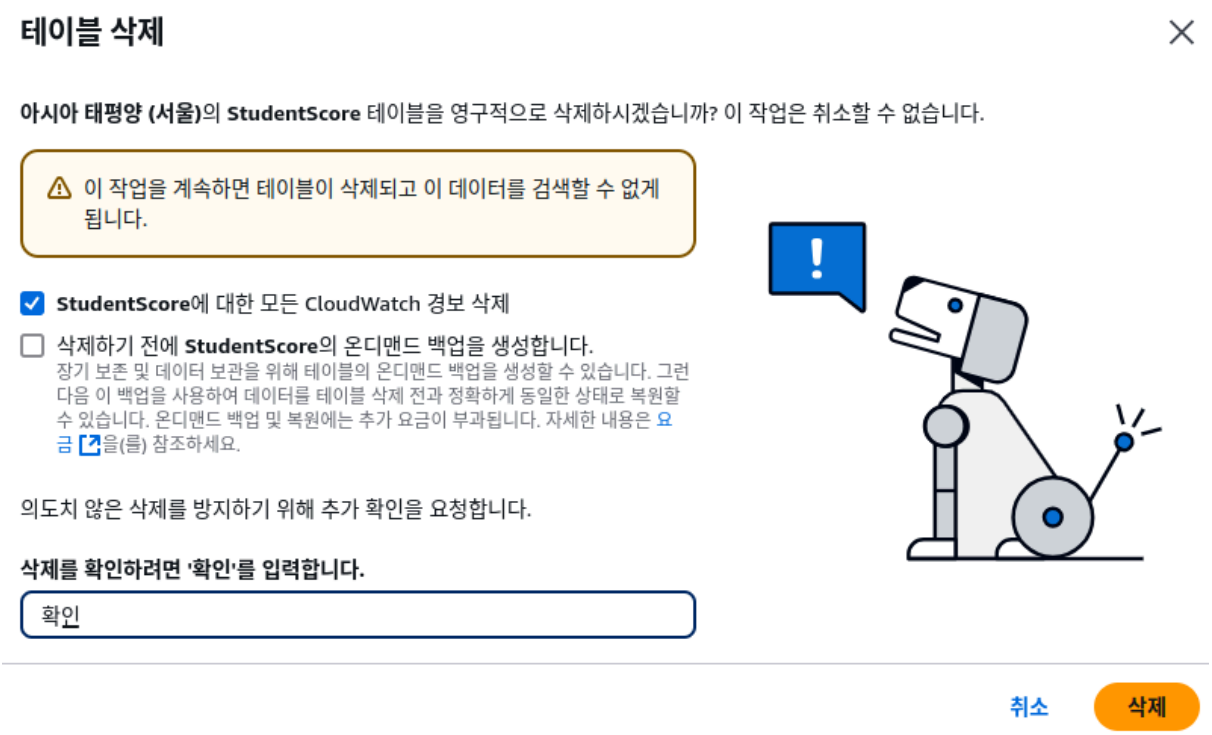


스캔과 쿼리

- 스캔: 전체 데이터를 가져온 이후에 필터링하는 방식
- 쿼리: 키 값을 이용해서 데이터를 찾는 방식

11. DynamoDB 테이블 삭제

- **테이블 상세 페이지 > 작업 > 테이블 삭제**



- **테이블 삭제**